PARTE A – CONSULTA

Bloque 1 - Power BI y Grafana

¿Qué es Grafana? Describan sus componentes: data sources, panels, dashboards, variables y alerting.

Grafana es una plataforma inicialmente gratuita que sirve para conectar con casi cualquier base de datos y generar dashboards intuitivos e interactivos en base a los mismos.

Data Sources: Es el lugar donde se encuentran los datos que se van a conectar con Grafana.

Panels: Es un objeto individual que se encarga de mostrar un dato en específico.

Dashboards: Es el tablero completo en el que se permite visualizar un conjunto de paneles.

Variables: Son filtros que permiten visualizar la información desde diferentes contextos, sin tener que volver a crear más paneles.

Alertas: Son aquellas señales que muestra el sistema cuando los datos toman una postura importante de analizar.

Diferencien Grafana OSS de Grafana Cloud. Elaboren una tabla comparativa Power BI vs. Grafana con mínimo 8 criterios: propósito principal, usuario típico, conexión a los datos, modelo semántico y lenguaje (DAX frente a la consulta del origen), actualización y tiempo real, alertas, licenciamiento y costo, y control de acceso.

Criterio	Power BI	Grafana
propósito principal	Ver cómo va el negocio en cuanto ventas, ganancias, gastos, clientes. Es decir, mira el pasado y el presente.	Ver si la tecnología funciona, si el servidor está vivo o si la web está lenta.
usuario típico	El gerente, el administrador, el contador o el analista de ventas.	El programador, el encargado de los servidores o el técnico de soporte.
conexión a los datos	Copia los datos dentro de sí mismo para poder cruzarlos y analizarlos a fondo.	No guarda nada. Va a la base de datos, lee lo que hay en ese instante y lo muestra.
modelo semántico y lenguaje	Puede unir datos de Excel con datos de Facebook y SQL. Usa un lenguaje llamado DAX (como fórmulas de Excel avanzadas).	No junta datos de diferentes mundos. Usa el idioma nativo de la base de datos que le conectes (como SQL o PromQL).
Actualización y tiempo real	Lento. Los datos se actualizan un par de veces al día	Inmediato. El gráfico se mueve en vivo cada 2 o 5 segundos

alertas	Comerciales. Te manda un email que dice: "Felicidades, llegamos a la meta de ventas del mes".	De emergencia. Te despierta en la madrugada con un mensaje al celular: "¡Alerta! El servidor se está quemando y la app se cayó".
licenciamiento y costo	Es de Microsoft. Hay que pagar una mensualidad por cada persona que quiera ver los reportes.	Es Gratis y libre. Lo descargas, lo instalas en tu computadora y lo usas sin pagar nada
control de acceso	Se conecta con las cuentas de Windows de la empresa. Es muy fácil decidir qué empleado ve qué reporte	Usa sistemas de usuarios de programadores. Defines quién puede editar gráficos y quién solo verlos.

D3. Diferencien un dashboard operativo, uno analítico y uno estratégico. Clasifiquen cada KPI mínimo del proyecto (energía, yield y Performance Ratio, ahorro, disponibilidad y alarmas, porcentaje de datos válidos) e indiquen en cuál de las dos herramientas lo publicarían y por qué.

KPI	Tipo	Herramienta	Por qué
Disponibilidad y alarmas	Operativo	Grafana	Exige reaccionar en minutos y Grafana alerta sobre series de tiempo
% de datos válidos	Operativo y analítico	Grafana para vigilarlo, Power BI para el resumen diario	Una caída repentina indica un sensor o una comunicación fallando
Energía (kWh)	Analítico	Power BI (el valor del día también en Grafana)	Se compara por día, mes y dispositivo
Yield y Performance Ratio	Analítico	Power BI	Son cálculos con medidas DAX sobre datos agregados
Ahorro	Estratégico	Power BI	Es una cifra en dinero para directivos, sin urgencia de tiempo real

D4. Grafana con PostgreSQL: ¿cómo debe ser una consulta de series de tiempo y para qué sirven las macros $__timeFilter y $__timeGroup? Escriban la consulta de la potencia promedio por hora.

```
SELECT
 $__timeGroup(ts, '1h') AS time,
 AVG(p_ac_kw) AS potencia_promedio_kw
FROM silver.lectura_5min
WHERE $__timeFilter(ts)
GROUP BY 1
ORDER BY 1;
```

Grafana necesita la fecha (time), el valor numérico y los datos ordenados. Las macros __$timeFilter y __$timeGroup filtran por el periodo seleccionado y agrupan los datos por tiempo, permitiendo calcular la potencia promedio por hora en SolarBI, que recibe datos cada 5 minutos (Grafana Labs, 2026b).

D5. Alertas: comparen Grafana Alerting (alert rules y contact points) con las alertas de datos del servicio de Power BI. Propongan una regla de alerta útil para la planta y el canal por el que se notificaría.

Grafana permite configurar reglas de alerta y medios de notificación, revisándolas cada minuto. Power BI solo alerta sobre tarjetas, KPI y medidores de dashboards web cuando se actualizan los datos. Por eso, Grafana es más útil para detectar fallas de la planta a tiempo (Grafana Labs, 2026c; Microsoft, 2026a).

Regla propuesta	**Detalle**
Condición	La potencia promedio p_ac_kw es 0 durante 15 minutos seguidos
Horario	Entre las 9:00 y las 15:00, cuando hay sol
Qué indica	Falla del inversor o de la comunicación
Canal	Telegram y correo al operador de la planta

Bloque 2 - Gobernanza de datos (data governance)

G1. Definan data governance y diferéncienla de data management y de data quality. Tomen como referencia el marco DAMA-DMBOK.

La gobernanza define reglas y responsables; la gestión almacena, integra y cuida los datos; y la calidad garantiza que sean completos, correctos y sin duplicados. Según DAMA-DMBOK, la gobernanza dirige las demás áreas. En SolarBI, se refleja en los roles y el contrato de datos, el proceso ETL y PostgreSQL, y las reglas de limpieza de Silver (DAMA International, 2017).

G2. Expliquen los roles data owner, data steward y data custodian. Relaciónenlos con los roles del equipo en el proyecto (data engineer, data modeler, BI analyst y product owner).

El data owner es el responsable final de un conjunto de datos: decide quién lo usa y para qué. El data steward lo cuida en el día a día: mantiene las definiciones, las reglas de calidad y resuelve los problemas. El data custodian se encarga de lo técnico: almacenar, respaldar y aplicar los permisos que el owner definió (DAMA International, 2017).

Rol de gobernanza	Rol en SolarBI	Por qué
Data owner	Product owner	Decide qué KPI se publican y quién puede verlos
Data steward	Data modeler y BI analyst	Definen el esquema, el significado de cada KPI y las reglas de calidad
Data custodian	Data engineer	Administra PostgreSQL, el ETL, los respaldos y los accesos

G3. El contrato de datos (data contract) del proyecto como instrumento de gobernanza: ¿qué debe incluir (esquema, unidades, frecuencia, responsable, niveles de calidad)? ¿Qué le pasa al cuadro de mando si la fuente cambia un campo sin avisar?

Un contrato de datos es un acuerdo escrito entre quien produce los datos y quien los usa. Así todos saben qué esperar y cualquier cambio debe avisarse antes (Bitol, 2026).

Elemento	En SolarBI
Esquema	ts, dispositivo_id, p_ac_kw, irradiancia_wm2, temp_modulo_c, con su tipo de dato
Unidades	kW, W/m^2 y °C
Frecuencia	Una lectura cada 5 minutos
Responsable	Quién entrega los datos y a quién se le reporta un problema
Niveles de calidad	Rangos válidos y porcentaje mínimo de datos válidos

Si la fuente cambia un campo sin avisar, por ejemplo, renombra p_ac_kw, el ETL falla, la actualización de Power BI da error y Grafana muestra el panel sin datos. Si cambia la unidad de kW a W es peor: nada falla, pero la energía aparece mil veces mayor.

G4. Seguridad a nivel de fila (row-level security, RLS) en Power BI: roles y filtros DAX. Den el ejemplo de un responsable que solo ve su sitio. ¿Cuál es el mecanismo equivalente de control de acceso en Grafana (organizaciones, equipos y permisos por carpeta o dashboard)?

La seguridad a nivel de fila (RLS) de Power BI sirve para que todos abran el mismo informe, pero cada persona vea solo los datos que le corresponden. Se hace en Power BI Desktop creando roles y poniéndole a cada rol una condición escrita en DAX que dice qué filas puede ver. Después, en la versión web, se indica qué usuarios pertenecen a cada rol (Microsoft, 2026b). Por ejemplo, para que un responsable solo vea su sitio, se usa una condición que compara el correo del responsable del sitio con el correo de quien inició sesión:

[correo_responsable] = USERPRINCIPALNAME()

Grafana gratuito (OSS) no puede ocultar filas según el usuario. Lo que hace es controlar quién puede entrar a cada tablero, usando organizaciones (espacios separados), equipos (grupos de usuarios) y permisos de ver, editar o administrar por carpeta o tablero. Lo más parecido sería crear una carpeta por cada sitio y que solo la vea el equipo de ese sitio (Grafana Labs, 2026d).

G5. Ley 1581 de 2012 (protección de datos personales en Colombia): ¿los datos de SolarBI son personales? ¿Qué cambiaría si el tablero mostrara qué operador atendió cada alarma? Fijen la postura del equipo frente al manejo responsable de la información.

Según la Ley 1581, los datos de SolarBI no son personales porque describen equipos. Sin embargo, identificar al operador de una alarma sí lo sería. En ese caso, se debe informar el uso de los datos, obtener la autorización correspondiente y proteger su acceso. Por ello, proponemos usar solo los datos necesarios, códigos en lugar de nombres, acceso restringido y nunca guardar contraseñas en el repositorio (Congreso de Colombia, 2012).

Bloque 3 - Automatización de procesos ETL

E1. Diferencien ETL de ELT y ubiquen cada paso en las capas Bronze, Silver y Gold del proyecto. ¿Power Query es una herramienta ETL? Argumenten.

En ETL, los datos se transforman antes de guardarlos; en ELT, se guardan primero y se transforman después. SolarBI se parece a ELT: conserva los datos originales en Bronze, los limpia en Silver y los resume en Gold. Power Query es una herramienta ETL, pero carga datos en Power BI, no en PostgreSQL, por lo que no reemplaza run_etl.py (Databricks, 2026; Microsoft, 2026f).

E2. Comparen carga completa (full load) y carga incremental. ¿Qué significa que una carga sea idempotente y cómo se logra en fact_energia_dia con su clave primaria (fecha_key, dispositivo_key) y un UPSERT?

```
INSERT INTO dwh.fact_energia_dia
 (fecha_key, dispositivo_key, energia_kwh, pct_datos_validos)
VALUES (%s, %s, %s, %s)
ON CONFLICT (fecha_key, dispositivo_key)
DO UPDATE SET
 energia_kwh = EXCLUDED.energia_kwh,
 pct_datos_validos = EXCLUDED.pct_datos_validos;
```

En la carga completa se borra todo y se vuelve a cargar desde cero cada vez. Es fácil de hacer, pero se vuelve lenta cuando hay muchos datos. En la carga incremental solo se agrega lo nuevo o lo que cambió desde la última vez. Es más rápida, pero hay que tener más cuidado para no dejar datos por fuera o repetidos. Una carga es idempotente cuando da igual ejecutarla una vez o varias: el resultado siempre es el mismo. En la tabla fact_energia_dia esto se logra porque la clave primaria (fecha_key, dispositivo_key) no permite dos filas del mismo día y del mismo dispositivo. Además, se usa un UPSERT, que guarda la fila si no existe y, si ya existe, solo la actualiza. Así, aunque el ETL se ejecute dos veces, no se duplica nada (PostgreSQL Global Development Group, 2026).

E3. Elaboren una tabla comparativa de tres formas de automatizar un flujo: cron (o el Programador de tareas de Windows), Apache Airflow y la actualización programada de Power BI. Incluyan qué es, qué automatiza cada una y cuándo conviene.

En SolarBI, cron ejecuta el ETL a medianoche y Power BI actualiza el informe después. Airflow no es necesario por ahora, pero sería útil si aumentaran los procesos y sus dependencias (Apache Software Foundation, 2026; Microsoft, 2026d).

Criterio	Programador de tareas	Apache Airflow	Actualización programada de Power BI
Qué es	Programador del sistema operativo que ejecuta un comando a una hora fija	Plataforma de orquestación: los flujos se escriben en Python como tareas con dependencias (DAG)	Función del servicio de Power BI que vuelve a consultar las fuentes
Qué automatiza	Un comando, por ejemplo, python etl/run_etl.py	Flujos de varios pasos, con reintentos, registro y monitoreo	Solo la actualización del modelo del informe; no ejecuta el ETL
Cuando conviene	Un flujo simple en un solo equipo, como esta práctica	Muchos flujos que dependen unos de otros y necesitan vigilancia	Siempre que el informe esté publicado; con PostgreSQL local requiere un gateway
Ejemplo en SolarBI	0 0 * * * ejecuta el ETL cada medianoche	Un DAG: simulador, luego calidad, luego carga	Actualizar el informe a las 00:30, después del ETL

E4. Query folding en Power Query: ¿qué es, por qué mejora el rendimiento y cómo se verifica si un paso se está plegando hacia PostgreSQL?

El query folding convierte los pasos de Power Query en SQL para que PostgreSQL procese los datos y Power BI reciba solo lo necesario, mejorando el rendimiento. Se verifica con clic derecho en un paso y seleccionando “Ver consulta nativa”; si está habilitada, se puede consultar el SQL generado.

Bloque 4 - Internet de las Cosas (IoT)

I1. Describan la arquitectura IoT por capas: dispositivo, gateway (edge), red o broker, almacenamiento y aplicación. Dibujen cómo viaja una lectura del inversor hasta llegar a un visual del tablero, indicando en qué capa (Bronze, Silver, Gold) queda en cada momento.

Capa	Qué hace en SolarBI	Dónde queda el dato
Dispositivo	El inversor y sus sensores miden potencia, irradiancia y temperatura	Aún no se almacena

Gateway (edge)	Lee al inversor, arma el mensaje y lo envía	En tránsito
Red o broker	El broker MQTT recibe el mensaje y lo reparte a quien esté suscrito	En tránsito
Almacenamiento	PostgreSQL y archivos guardan las lecturas	Bronze, luego Silver, luego Gold
Aplicación	Grafana y Power BI muestran los datos	Grafana lee Silver; Power BI lee Gold



I2. Expliquen MQTT: modelo publish/subscribe, broker, topic y niveles de QoS (0, 1 y 2). Propongan la jerarquía de topics de la planta, por ejemplo pascualbravo/solar/<sitio>/<dispositivo>/<variable>.

pascualbravo/solar/<sitio>/<dispositivo>/<variable>

pascualbravo/solar/bloque-a/inv01/p_ac_kw

pascualbravo/solar/bloque-a/inv01/irradiancia_wm2

pascualbravo/solar/bloque-a/inv01/temp_modulo_c

pascualbravo/solar/bloque-a/inv01/estado

pascualbravo/solar/+/+/p_ac_kw (la potencia de todos los dispositivos)

pascualbravo/solar/bloque-a/# (todo lo de un sitio)

MQTT es un protocolo de mensajería liviano para IoT. Usa el modelo publish/subscribe: los dispositivos publican mensajes y las aplicaciones se suscriben a los que les interesan, sin conocerse entre sí. El broker es el servidor intermedio que recibe y reparte los mensajes. El topic es una etiqueta jerárquica, separada por barras, que indica de qué trata cada mensaje. Los niveles de QoS definen la garantía de entrega: 0, a lo sumo una vez (puede perderse); 1, al menos una vez (puede repetirse); 2, exactamente una vez (más lento). Para SolarBI conviene QoS 1, porque los duplicados los elimina la regla de calidad de Silver (OASIS, 2014).

I3. Describan una pila abierta típica de monitoreo IoT: Mosquitto, Telegraf, una base de series de tiempo (InfluxDB o TimescaleDB) y Grafana. ¿Qué papel cumple cada componente y dónde encajaría PostgreSQL?

Es un conjunto de cuatro programas gratuitos y de código abierto que procesan los datos desde que llegan del dispositivo hasta que se visualizan. PostgreSQL, con TimescaleDB, almacena las lecturas y también el almacén de datos dwh de Power BI. Así, Grafana y Power BI consultan la misma base de datos (Eclipse Foundation, 2026; InfluxData, 2026).

Componente	Papel
Mosquitto	Broker MQTT: recibe los mensajes del gateway y los reparte
Telegraf	Agente recolector: se suscribe a los topics y escribe las lecturas en la base
InfluxDB o TimescaleDB	Base de series de tiempo: guarda y consulta rápido millones de lecturas con fecha
Grafana	Visualización y alertas sobre esas lecturas

I4. Procesamiento por lotes (batch) frente a flujo continuo (streaming). La telemetría llega cada 5 minutos y fact_energia_dia tiene granularidad diaria: ¿qué se gana y qué se pierde al agregar? ¿Qué herramienta atiende mejor cada granularidad?

El batch procesa datos acumulados periódicamente, mientras que el streaming procesa cada dato al llegar. Resumir las lecturas en valores diarios agiliza las consultas, pero pierde detalles de fallas breves. Por eso, Grafana se usa para monitorear lecturas cada 5 minutos y Power BI para analizar datos diarios y tomar decisiones (Kleppmann, 2017).

V1. Diferencien Git de GitHub y definan repository, commit, branch y pull request. Un archivo .pbix es binario: ¿qué problema genera en Git y qué resuelve el formato Power BI Project (.pbip)? ¿Cómo se versiona un dashboard de Grafana?

Git es un sistema de control de versiones que funciona en el computador y guarda el historial de cambios de un proyecto. GitHub es una plataforma en la nube que aloja repositorios Git y facilita el trabajo en equipo (Chacon y Straub, 2014; GitHub, 2026).

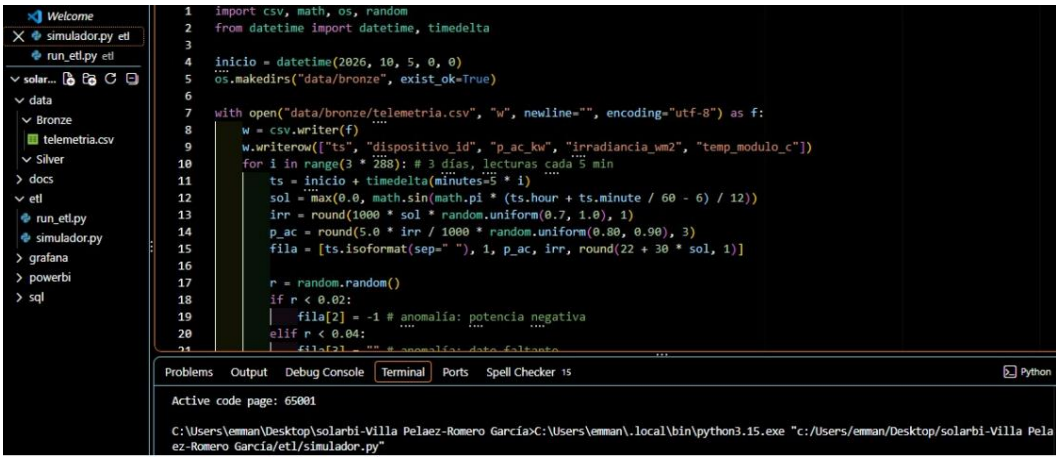
Término	Definición
Repository	Carpeta del proyecto con todo su historial de cambios
Commit	Foto de los cambios en un momento, con autor, fecha y mensaje
Branch	Línea de trabajo paralela para cambiar algo sin afectar la principal
Pull request	Propuesta en GitHub para revisar e integrar una rama en otra

El archivo .pbix es binario, dificulta comparar cambios en Git y aumenta el tamaño del repositorio. El formato. pbip guarda informes y modelos en archivos de texto, facilitando el control de versiones. En Grafana, los tableros se exportan en JSON y se guardan en grafana/dashboard.json para registrar sus cambios en Git.

PARTE B PRACTICA GUIADA

Paso 1 - Bronze: datos crudos

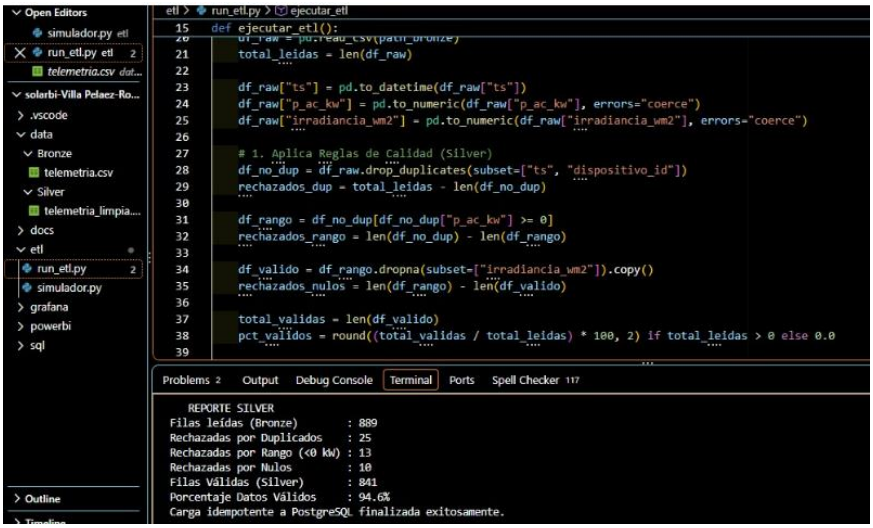
figura 1. Captura del archivo csv generado en la capa Bronze (data/bronze/telemetria.csv).



Nota. Ejecución de etl/simulador.py para generar la capa Bronze (data/bronze/telemetria.csv) con 3 días de telemetría cada 5 minutos (864 registros), incluyendo datos nulos, potencias negativas y duplicados.

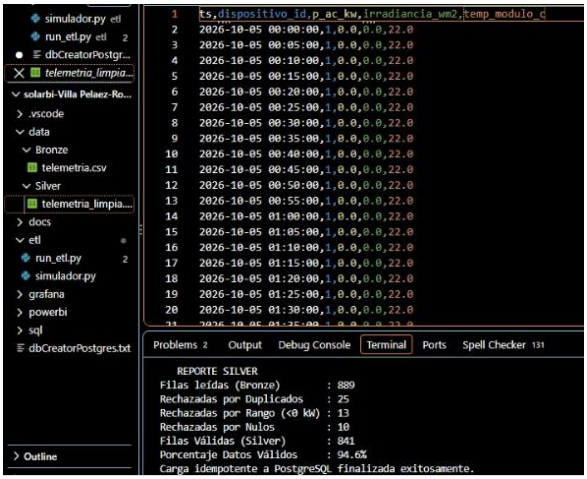
Paso 2 - Silver: reglas de calidad

Figura 2: Salida en consola de la ejecución del pipeline ETL (etl/run_etl.py).



Nota. Ejecución del ETL en Python: de 889 filas Bronze se filtraron 25 duplicados, 13 valores fuera de rango y 10 lecturas incompletas. Silver obtuvo 841 registros limpios, con un SLA de calidad del 94,6 %. Se confirmó la carga idempotente en PostgreSQL (silver.lectura_5min y dwh.fact_energia_dia).

Figura 3. Estructura del dataset depurado en la capa silver (data/silver/telemetria_limpia.csv).



Nota. Archivo CSV de la capa Silver con datos limpios y validados, sin nulos ni inconsistencias, conservando lecturas cada 5 minutos por dispositivo. Sirve como respaldo local y fuente confiable para generar las métricas diarias de la capa Gold.

Figura 4. Creación de la base de datos principal solarbi_db con tabla silver.lectura_5min

```
CREATE DATABASE solarbi_db;
\c solarbi_db;

CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS dwh;

CREATE TABLE IF NOT EXISTS silver.lectura_5min (
    ts TIMESTAMP NOT NULL,
    dispositivo_id INT NOT NULL,
    p_ac_kw NUMERIC(8,3),
    irradiancia_wm2 NUMERIC(8,1),
    temp_modulo_c NUMERIC(5,1),
    PRIMARY KEY (ts, dispositivo_id)
);

CREATE TABLE IF NOT EXISTS dwh.fact_energia_dia (
    fecha DATE NOT NULL,
    dispositivo_id INT NOT NULL,
    energia_kwh NUMERIC(10,3),
    pct_datos_validos NUMERIC(5,2),
    PRIMARY KEY (fecha, dispositivo_id)
);

\q
```

Nota. El código se ejecuta una sola vez, antes del primer proceso ETL, para preparar la infraestructura de almacenamiento del proyecto. Se ejecuta en la consola interactiva de PostgreSQL (psql), desde PowerShell o CMD de Windows.

Su función es crear la base de datos solarbi_db y conectarse a ella. Luego, crea los esquemas silver, para almacenar datos detallados y limpios, y dwh, correspondiente a la capa Gold, destinada al análisis y los resúmenes ejecutivos. Después, crea las tablas silver.lectura_5min, que almacena las lecturas de los paneles solares y evita duplicados

mediante una llave primaria compuesta, y dwh.fact_energia_dia, que registra la energía diaria acumulada en kWh y la calidad de los datos. Finalmente, \q cierra la consola de PostgreSQL y regresa a la terminal de Windows.

Figura 5. Control de duplicados con df_no_dup

```
df_no_dup = df_raw.drop_duplicates(subset=["ts", "dispositivo_id"])
rechazados_dup = total_leidas - len(df_no_dup)
```

Nota. El código elimina las lecturas duplicadas de los paneles solares, identificándolas mediante la fecha/hora (ts) y el código del panel (dispositivo_id). Conserva la primera lectura y descarta las repetidas, evitando contabilizar dos veces la energía generada y garantizando que los datos de la capa Silver sean únicos para cada dispositivo y momento.

Figura 6. Rango físico válido ante potencia y registros de valores mayores e iguales a 0.

```
df_rango = df_no_dup[df_no_dup["p_ac_kw"] >= 0]
rechazados_rango = len(df_no_dup) - len(df_rango)
```

Nota. El código valida la potencia activa (p_ac_kw) y conserva únicamente los registros con valores mayores o iguales a cero, descartando los datos negativos que pueden deberse a fallas o descalibraciones de los sensores. Además, contabiliza los registros inválidos, evitando errores en los cálculos diarios de energía y garantizando que solo los datos válidos continúen hacia el almacén de datos.

Figura 7: Datos faltantes y/o nulos a descartar

```
df_valido = df_rango.dropna(subset=["irradiancia_wm2"]).copy()
rechazados_nulos = len(df_rango) - len(df_valido)

total_validas = len(df_valido)
pct_validos = round((total_validas / total_leidas) * 100, 2) if total_leidas > 0 else 0.0
```

Nota. El código identifica y elimina los registros que no tienen información de irradiancia solar, ya sea por valores nulos o datos faltantes. Además, contabiliza las filas eliminadas y calcula el porcentaje final de datos válidos (SLA de calidad), garantizando que las métricas de eficiencia se calculen con información completa.

Figura 8: Carga en postgreSQL e indempotencia para tabla silver.



```
with engine.begin() as conn:
    for _, row in df_valido.iterrows():
        query_silver = text("""
            INSERT INTO silver.lectura_5min (ts, dispositivo_id, p_ac_kw, irradiancia_wm2, temp_modulo_c)
            VALUES (:ts, :dispositivo_id, :p_ac_kw, :irradiancia_wm2, :temp_modulo_c)
            ON CONFLICT (ts, dispositivo_id) DO UPDATE SET
                p_ac_kw = EXCLUDED.p_ac_kw,
                irradiancia_wm2 = EXCLUDED.irradiancia_wm2,
                temp_modulo_c = EXCLUDED.temp_modulo_c;
        """)
        conn.execute(query_silver, {
            "ts": row["ts"],
            "dispositivo_id": int(row["dispositivo_id"]),
            "p_ac_kw": float(row["p_ac_kw"]),
            "irradiancia_wm2": float(row["irradiancia_wm2"]),
            "temp_modulo_c": float(row["temp_modulo_c"])
        })
```

Nota. El código almacena las lecturas válidas de cada 5 minutos en la tabla silver.lectura_5min, conservando el detalle técnico del parque solar durante el día. Utiliza ON CONFLICT (ts, dispositivo_id) DO UPDATE para actualizar los registros existentes con los datos más recientes, evitando errores y duplicados.

Figura 9: Carga en postgreSQL e idempotencia etapa gold dwh.fact_energia_dia.



```
    df_valido["energia_intervalo_kwh"] = df_valido["p_ac_kw"] * (5 / 60)
    df_valido["fecha"] = df_valido["ts"].dt.date

    df_gold = df_valido.groupby(["fecha", "dispositivo_id"]).agg(
        energia_kwh=("energia_intervalo_kwh", "sum")
    ).reset_index()
    df_gold["pct_datos_validos"] = pct_validos

    with engine.begin() as conn:
        for _, row in df_gold.iterrows():
            query_gold = text("""
                INSERT INTO dwh.fact_energia_dia (fecha, dispositivo_id, energia_kwh, pct_datos_validos)
                VALUES (:fecha, :dispositivo_id, :energia_kwh, :pct_datos_validos)
                ON CONFLICT (fecha, dispositivo_id) DO UPDATE SET
                    energia_kwh = EXCLUDED.energia_kwh,
                    pct_datos_validos = EXCLUDED.pct_datos_validos;
            """)
            conn.execute(query_gold, {
                "fecha": row["fecha"],
                "dispositivo_id": int(row["dispositivo_id"]),
                "energia_kwh": float(round(row["energia_kwh"], 3)),
                "pct_datos_validos": float(row["pct_datos_validos"])
            })

    print("Carga idempotente a PostgreSQL finalizada exitosamente.")

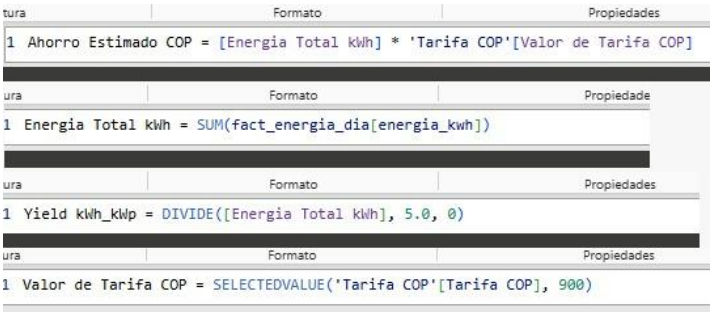
if __name__ == "__main__":
    ejecutar_etl()
```

El código convierte la potencia de cada intervalo de 5 minutos en energía (kWh), multiplicándola por \(5/60\) de hora. Luego, agrupa las mediciones por día y panel solar para calcular la energía total generada y el porcentaje de datos válidos. Finalmente, guarda el resumen en dwh.fact_energia_dia, donde los tableros de gestión y Power BI consultan la información, evitando duplicados mediante la resolución de conflictos por fecha y dispositivo.

La idempotencia permite ejecutar el proceso varias veces sin duplicar datos ni alterar el resultado final. En el proyecto, se comprobó al ejecutar nuevamente python etl/run_etl.py, manteniendo los 841 registros en Silver y sin errores por claves duplicadas. Esto demuestra que el proceso puede Re ejecutarse de forma segura.

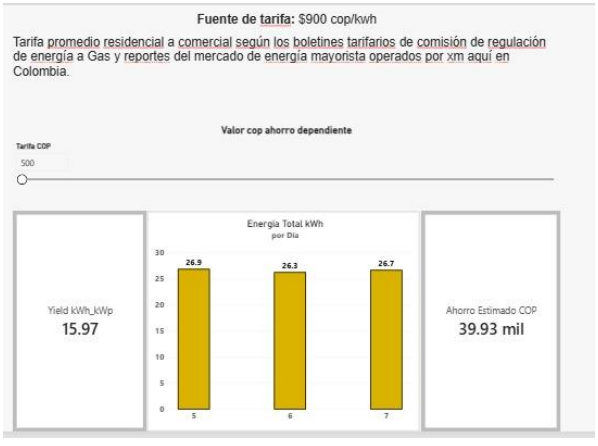
Para automatizar la ejecución diaria a las 00:00 horas, se puede utilizar cron en Linux y macOS, mediante una expresión que ejecuta el script ETL y registra los resultados en un archivo de log. En Windows, se utiliza el Programador de tareas (Task Scheduler), configurando la ruta del proyecto y un script que ejecute el proceso automáticamente.

Figura 10. Implementación de indicadores de rendimiento en Power BI mediante DAX



Nota. Se implementaron tres indicadores en DAX para el panel gerencial. [Energía Total kWh] suma la energía producida; [Yield kWh_kWp] calcula el rendimiento del parque solar respecto a su capacidad instalada de 5,0 kWp mediante DIVIDE, evitando errores de división por cero; y [Ahorro Estimado COP] calcula el ahorro económico multiplicando la energía generada por la tarifa configurada, permitiendo actualizar las estimaciones financieras en tiempo real.

Figura 11. Diseño del tablero gerencial en Power BI para el análisis de generación solar



Nota. El tablero gerencial desarrollado en Power BI integra los datos de la capa Gold (dwh. fact_energia_dia) mediante un parámetro interactivo de tarifa, tarjetas de indicadores clave como el rendimiento (kWh/kWp) y el ahorro acumulado en pesos colombianos, y un gráfico

de columnas que muestra la generación diaria. Además, incluye un texto explicativo sobre el costo unitario.

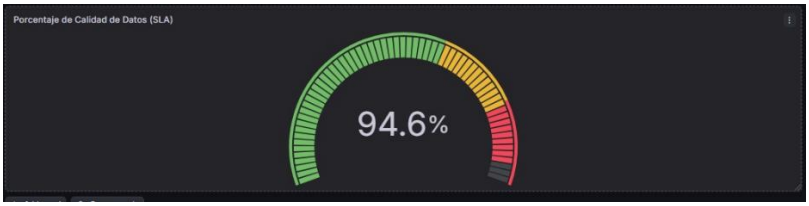
Esta distribución permite analizar el rendimiento técnico y el impacto financiero del parque solar en tiempo real. Los indicadores y el gráfico se actualizan dinámicamente según la tarifa seleccionada y el rango de fechas evaluado.

Figura 12. Monitoreo de potencia activa e irradiancia solar mediante series de tiempo



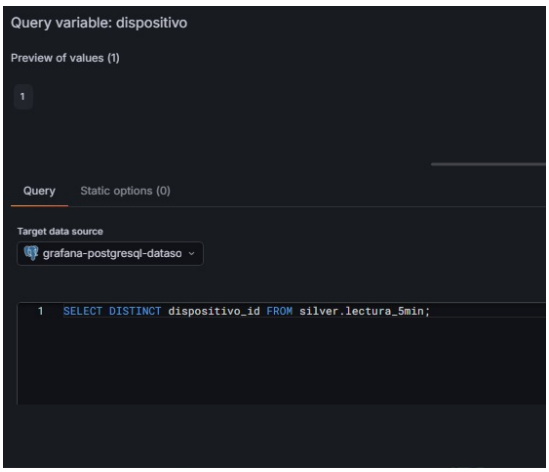
Nota. Se creó un panel de series de tiempo en Grafana conectado a la tabla silver.lectura_5min mediante una consulta SQL con la macro $__timeFilter(ts), que filtra los datos por fecha y hora. La gráfica utiliza dos ejes Y: el izquierdo para la potencia activa y el derecho para la irradiancia solar, permitiendo comparar ambas variables y visualizar las lecturas cada 5 minutos de forma eficiente desde PostgreSQL.

Figura 13. Monitoreo de la calidad e integridad de los datos mediante un indicador en Grafana



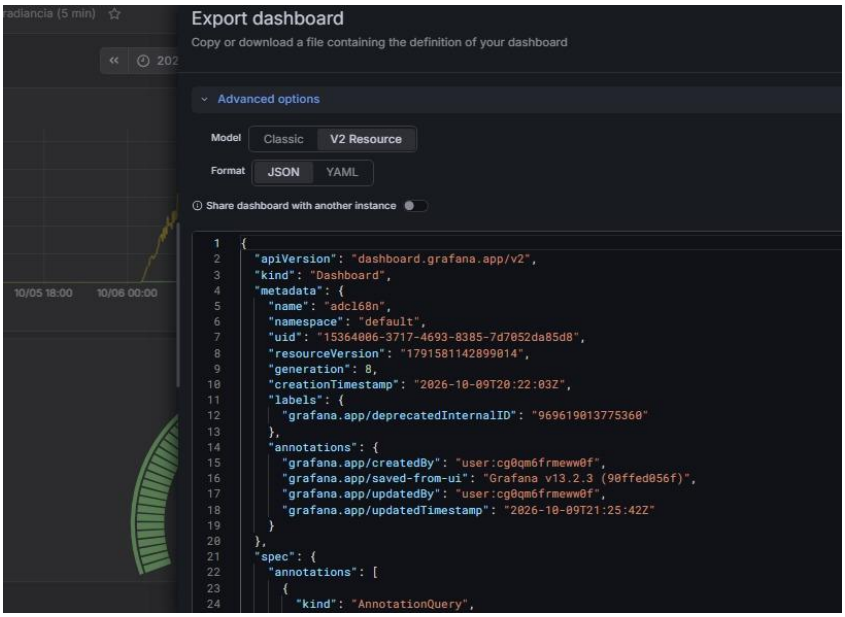
Nota. Se implementó un panel tipo Gauge en Grafana que muestra el porcentaje de datos válidos (pct_datos_validos), calculado durante el proceso ETL y almacenado en dwh.fact_energia_dia. El indicador evalúa la calidad de los datos considerando nulos, duplicados y lecturas fuera de rango, utilizando un promedio y una escala de colores tipo semáforo para visualizar el cumplimiento del nivel de servicio.

Figura 14. Configuración de filtros dinámicos por dispositivo solar en Grafana



Nota. Se configuró la variable global $dispositivo en Grafana mediante una consulta SQL que obtiene los dispositivos registrados. Esta variable permite filtrar los paneles por un inversor específico o visualizar los datos de toda la planta, facilitando el análisis interactivo de la información.

Figura 15. Exportación y versionamiento del dashboard de Grafana para su despliegue



Nota. Se exportó el tablero de Grafana en el archivo grafana_dashboard.json, que contiene las consultas SQL, la distribución de los paneles y las reglas de formato. La opción Export for sharing externally facilita su portabilidad al separar la configuración de las credenciales locales. Al incluirlo en el repositorio del proyecto, se permite restaurar, versionar y desplegar el tablero en distintos entornos, garantizando la trazabilidad y facilitando la integración continua.

Fuentes

• Apache Software Foundation. (2026). Apache Airflow documentation. airflow.apache.org

• Bitol. (2026). Open Data Contract Standard. Linux Foundation. bitol-io.github.io

• Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. git-scm.com/book

• Congreso de Colombia. (2012). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. funcionpublica.gov.co

• DAMA International. (2017). DAMA-DMBOK: Data Management Body of Knowledge (2.ª ed.). Technics Publications.

• Databricks. (2026). Medallion architecture. databricks.com

• Eclipse Foundation. (2026). Eclipse Mosquitto: an open source MQTT broker. mosquitto.org

• Few, S. (2006). Information Dashboard Design: The Effective Visual Communication of Data. O'Reilly.

• GitHub. (2026). Documentación de GitHub. docs.github.com

• Grafana Labs. (2026a). Dashboards. grafana.com/docs

• Grafana Labs. (2026b). PostgreSQL data source. grafana.com/docs

• Grafana Labs. (2026c). Grafana Alerting. grafana.com/docs

• Grafana Labs. (2026d). Roles and permissions. grafana.com/docs

• Grafana Labs. (2026e). Dashboard JSON model. grafana.com/docs

• InfluxData. (2026). Telegraf documentation. docs.influxdata.com

• Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly.

• Microsoft. (2026a). Alertas de datos en el servicio Power BI. learn.microsoft.com

• Microsoft. (2026b). Seguridad de nivel de fila (RLS) con Power BI. learn.microsoft.com

• Microsoft. (2026c). Conceptos básicos del plegado de consultas. learn.microsoft.com

• Microsoft. (2026d). Configuración de la actualización programada. learn.microsoft.com

• Microsoft. (2026e). Proyectos de Power BI Desktop (PBIP). learn.microsoft.com

• Microsoft. (2026f). ¿Qué es Power Query? learn.microsoft.com

• Microsoft. (2026g). Arquitectura de referencia de Azure IoT. learn.microsoft.com

• OASIS. (2014). MQTT Version 3.1.1. docs.oasis-open.org